

WIPRO CAPSTONE PROJECT



Smart Energy Smart-Meter Pulse Counter & Analytics Agent

Final project report | Stages 1-6 | Source code appendix

Prepared by: **Smrutiranjana Behera**

Software-only Linux project | C++ application + C kernel driver

Ubuntu Linux 24.04 ARM64 in UTM | 4 October 2026



SIMULATED PULSES / IOCTL COMMUNICATION / ENERGY ANALYTICS

Contents

Section	Contents
Executive Summary	Project purpose, architecture, tested outcomes, requirements coverage and scope notes
Stage 1 - Introduction	Problem, objectives, scope, expected outcome and project background
Stage 2 - Requirements	PRD, functional/non-functional requirements, modules, timeline and acceptance criteria
Stage 3 - Design	Architecture, data flow, interfaces, formulas, UML, state machine and repository structure
Stage 4 - Prototype	Implemented modules, Ubuntu environment, demonstration and issues resolved
Stage 5 - Testing	Test plan, recorded results, observed issues and eight analytics unit checks
Stage 6 - Final Report	Final project summary, implementation and demonstration record
Appendix A	Build/run/test quick reference and sample test output
Appendix B	Complete source and build files with file-by-file explanations
Appendix C	Submission notes and GitHub repository

The report distinguishes planned scope in the early stage documents from features verified in the final implementation.

Executive Summary

This individual capstone implements a software-only smart-meter model on Linux. A C kernel module registers a virtual miscellaneous character device at `/dev/smart_meter`. A C++17 command-line program simulates pulse inputs, communicates with the driver using IOCTL requests, and calculates estimated energy, average power and cost.

The repository contains six staged documents, architecture/UML diagrams, the C/C++ implementation, build instructions, a manual integration record and eight passing analytics unit checks. No physical meter is connected, and the program does not persist reading history.

Project at a glance

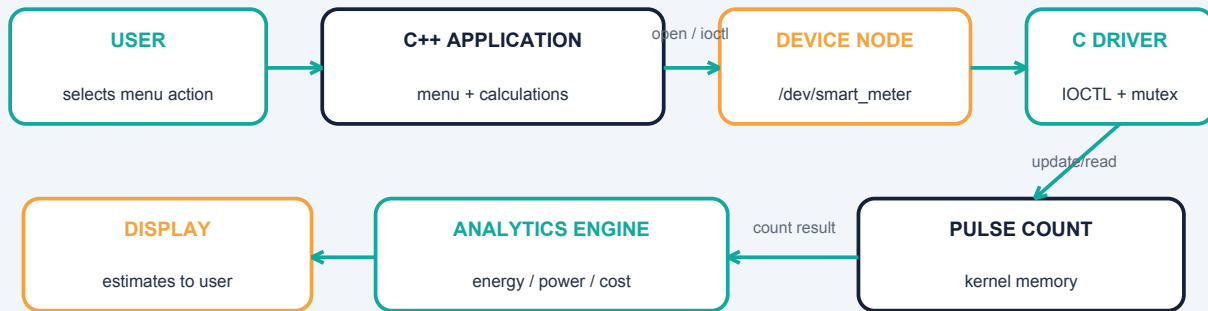
Item	Delivered project
Domain	IoT, Embedded Systems and Virtual Sensors
Operating system	Ubuntu Linux 24.04 ARM64 in UTM on Apple Silicon
Implementation	C Linux kernel module; C++17 user-space application
Device interface	<code>/dev/smart_meter</code> with IOCTL commands
Meter constant / sample rate	1,000 pulses per kWh / INR 8 per kWh
Input and storage	Simulated pulses; cumulative count held in driver memory only while loaded

Delivered capabilities and scope boundaries

Area	Status in the submitted implementation
Pulse simulation, count, read, reset	Implemented and manually demonstrated.
Energy, average power, estimated cost	Implemented; power uses elapsed time between readings.
Unit tests	Eight C++ analytics checks passed. They do not load the driver.
Usage classification / persistent data logging	Not implemented. Listed as possible future work, not as current features.
Physical meter or sensor	Not connected; all pulse inputs are simulated.
Configuration	Meter constant and sample tariff are currently fixed constants in the application.

Architecture overview

SIMULATED PULSE -> LINUX DRIVER -> C++ ANALYTICS



Counter is volatile: unloading the driver or restarting Linux clears it.

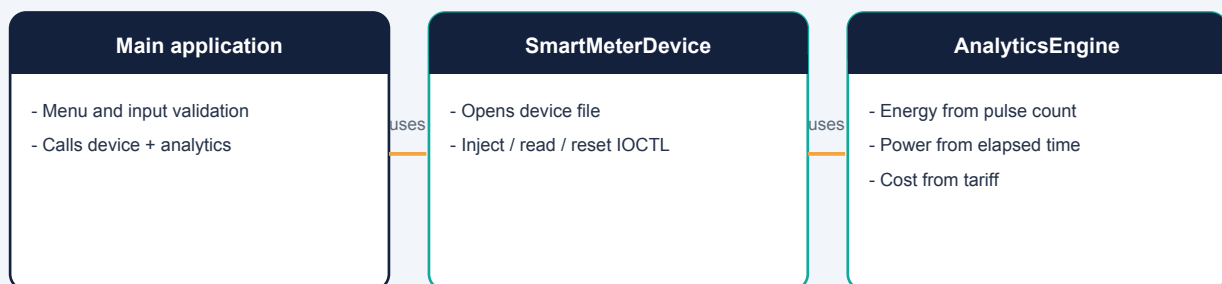
Analytics example

Energy (kWh) = cumulative pulses / 1,000. Estimated cost = energy x INR 8 per kWh. Average power (W) = energy change (kWh) x 3,600,000 / elapsed seconds.

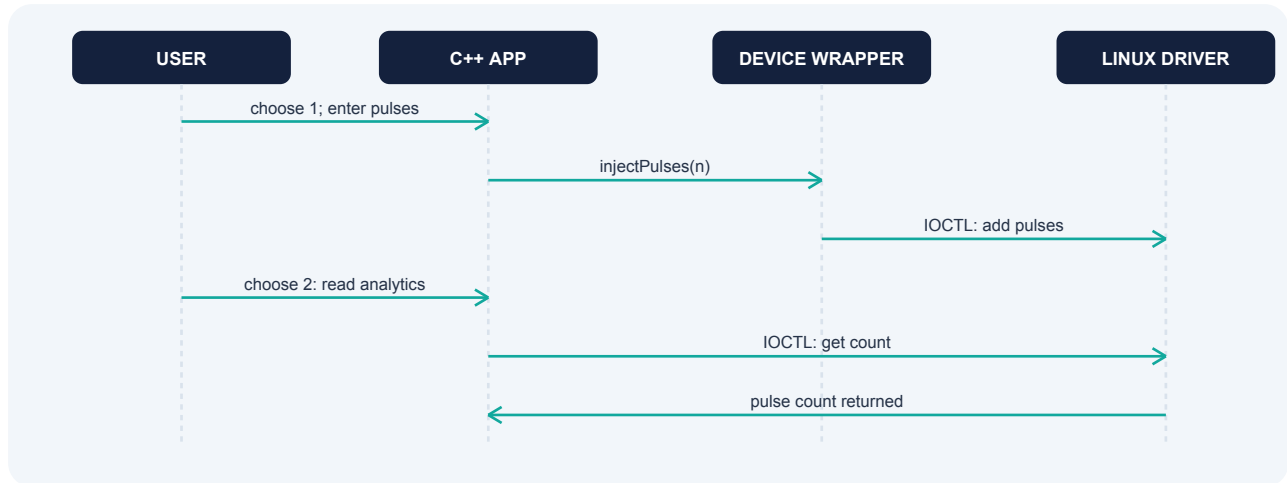
Reading	Cumulative pulses	Energy	Estimated cost	Evidence note
1	100	0.100 kWh	INR 0.800	Recorded manual test
2	101	0.101 kWh	INR 0.808	Recorded manual test
3	201	0.201 kWh	INR 1.608	Calculated illustration; not a recorded integration test

The application prints average power for subsequent readings. Its numerical value varies with how many seconds elapsed, so a fixed power number is not appropriate in this worked example.

Design diagrams



Kernel driver is C; it is not represented as a C++ class.



Requirements coverage and scope reconciliation

Stages 1-3 document the intended scope and design. The completed implementation is narrower and should be evaluated against the delivered behavior below. Early proposals for load profiles, alerts, configurable rates, usage categories and persistent logging were planning ideas; they are not represented as completed code features.

Requirement / goal	Final outcome
Linux driver and device interface	Implemented with a C miscellaneous character device, IOCTL operations and mutex-protected pulse counter.
C++ application to generate / read / reset pulses	Implemented in the menu-driven command-line application.
Energy and cost calculations	Implemented using 1,000 pulses/kWh and sample INR 8/kWh constants.
Average power estimate	Implemented from pulse-count change and elapsed time between readings.
Configurable meter constant / tariff	Not interactive/configurable in the delivered CLI; current values are fixed constants.
Usage categories, alerts and persistent history	Not implemented; future scope only.
No external hardware	Met; all input is simulated.

Stage 1 - Project Introduction

Project Title

Official project name: Smart Energy Smart-Meter Pulse Counter & Analytics Agent Technical title: Linux-Based Virtual Smart Meter Device Driver and Energy Analytics Agent

Project Domain

IoT, Embedded Systems & Virtual Sensors

Project Type

Individual, software-only Linux project. Meter pulses and sensor readings are simulated in software; no physical hardware is required.

Background

Electricity meters can represent energy use as pulses. A meter constant defines how many pulses correspond to one kilowatt-hour (kWh). Counting those pulses makes it possible to estimate energy consumption and, using the time between pulses, estimate power.

This project models that process entirely in software. A virtual meter generates simulated pulse events. A Linux device driver provides a kernel-to-user-space interface, and a C++ application processes the events and presents energy analytics.

Problem Statement

The project must demonstrate Linux device-driver concepts, system programming, and C++ development without requiring physical meter hardware. A software model is needed to generate repeatable meter events and show how Linux software can acquire, process, and analyze them.

Motivation

- Learn how a Linux application communicates with a device driver.
- Practice C++ programming through a practical energy-monitoring example.
- Explore virtual sensors and event-based data processing.
- Make the project easy to demonstrate using repeatable simulated input.

Project Goal

Build a Linux-based virtual smart-meter system that processes simulated meter pulses and presents estimated energy usage and related analytics.

Objectives

- Generate virtual meter pulses using configurable simulated load patterns.
- Provide a Linux device-driver interface for simulated meter events.
- Read and process those events in a C++ user-space application.
- Count pulses and calculate energy using a configurable meter constant.
- Estimate power from the time between pulses.
- Display virtual sensor readings and usage statistics.

- Record readings and report configurable high-usage alerts.
- Document and demonstrate the work through the instructor's six project stages.

Proposed Solution

The virtual meter generates timestamped pulses for low, normal, or high simulated loads. The Linux driver exposes meter data to the user-space application. The C++ application counts pulses, estimates energy and power, and sends the calculated readings to virtual sensor and analytics modules. A command-line interface displays the results, while the logger stores readings and alerts.

The exact driver interface and event format will be designed during Stage 3.

Project Scope

Included

- Software-generated smart-meter pulses.
- A Linux virtual device-driver component.
- A C++ user-space application.
- Pulse counting and configurable pulse-to-energy calculation.
- Estimated power, usage statistics, logging, and threshold alerts.
- Documentation, testing, version control, and project demonstrations.

Excluded

- Physical electricity meters or sensors.
- Arduino, ESP32, or other hardware requirements.
- Measuring actual household electricity consumption.
- Utility billing or production-grade metering.

Expected Outcome

A working demonstration in which simulated meter pulses pass through a Linux device interface to a C++ application. The application will display pulse count, estimated energy, estimated power, and selected usage analytics. The final submission will include source code, documentation, test evidence, diagrams, and a presentation.

Applications

- Teaching Linux device-driver and user-space communication concepts.
- Demonstrating C++ system programming with simulated event data.
- Exploring how virtual sensor readings can feed energy analytics.
- Prototyping energy-monitoring calculations using repeatable software input.

This project is educational; it does not measure real electrical consumption.

Planned Technology Stack

- Operating system: Linux
- Kernel driver: C
- User-space application: C++
- Build system: Make or CMake, to be selected during Stage 3
- Version control: Git and GitHub

- Development tools: GCC/G++, Linux kernel headers, GDB, and Linux system-inspection tools

Linux kernel drivers are written in C. The main application will be written in C++.

High-Level Architecture

Rendered design diagrams are presented in the consolidated architecture section of this report.

Limitations

- Pulses are simulated, so results cannot be verified against a physical meter.
- Power is estimated from simulated pulse timing and the configured meter constant.
- Cost estimates depend on a configured tariff and are not utility bills.
- Driver development requires a compatible Linux environment and kernel headers.
- This educational project is not intended for billing or production use.

Future Scope

Possible improvements include more simulated load profiles, configurable tariff schedules, historical reports, and additional analytics. Connecting physical devices would be a separate scope change; this project remains software-only.

Next Stage

Stage 2 will define the functional and non-functional requirements, project modules, features, deliverables, and development timeline.

Stage 2 - Project Requirements and Development Plan

Project Requirements Document (PRD)

Project Title

Linux-Based Virtual Smart-Meter Pulse Counter and Energy Analytics Agent

Project Objective

Build a software-only smart-meter prototype for Linux. A C Linux kernel module will expose a virtual meter device. A C++ application will send simulated pulse events to the device, read the accumulated count, and calculate estimated energy use, power, and cost.

Problem Statement

A physical smart meter counts electrical energy pulses and provides readings for monitoring and analysis. This project models that flow in software so that Linux device-driver concepts and C/C++ system programming can be learned without connecting physical meter hardware.

Intended Users

- Students learning Linux device drivers and system programming.
- Evaluators who need to run and observe the prototype in a Linux environment.

Project Scope

Included

- A Linux kernel module written in C.
- A virtual character device named `/dev/smart_meter`.
- A C-compatible header defining the driver's control interface.
- A C++ command-line application that simulates meter pulses and requests readings.
- Energy, estimated power, and estimated cost calculations.
- Build and run instructions for Ubuntu Linux.
- Documentation, diagrams, test records, and GitHub source code.

Excluded

- Physical meters, sensors, microcontrollers, and external hardware.
- Python, Java, or other programming languages for project implementation.
- Utility billing, legally valid meter readings, or production deployment.
- A graphical user interface or cloud service.

Functional Requirements

- FR-01: The C kernel module shall register a virtual character device named `/dev/smart_meter`.
- FR-02: The driver shall maintain a non-negative 64-bit cumulative pulse count.
- FR-03: The driver shall provide an `ioctl` operation to add a specified number of simulated pulses.
- FR-04: The driver shall provide an `ioctl` operation to read the current pulse count.
- FR-05: The driver shall provide an `ioctl` operation to reset the count for a new demonstration.

- FR-06: The driver shall protect shared counter data against concurrent access.
- FR-07: The C++ application shall open and communicate with `/dev/smart_meter`.
- FR-08: The application shall let the user generate simulated pulse events and display the total pulse count.
- FR-09: The application shall convert pulses into estimated energy using a configurable meter constant, initially 1000 pulses per kWh.
- FR-10: The application shall estimate power from pulse changes over a measured time interval.
- FR-11: The application shall estimate cost using a configurable sample tariff.
- FR-12: The application shall report clear errors when the device is unavailable or an operation fails.

Non-Functional Requirements

- NFR-01 - Language: Project implementation source code shall use only C and C++.
- NFR-02 - Operating system: Build and run the project on Linux, using the Ubuntu virtual machine.
- NFR-03 - Reliability: Reject invalid pulse values and report failed system calls clearly.
- NFR-04 - Safety: Loading and unloading the kernel module require administrator privileges. Do not use this prototype for real billing or electrical control.
- NFR-05 - Maintainability: Keep driver, shared interface definitions, C++ application, and documentation in separate, clearly named files or folders.
- NFR-06 - Usability: Provide simple commands for building, loading, running, and unloading the prototype.
- NFR-07 - Version control: Commit each completed stage and push it to the project's GitHub repository.

Planned Modules

- Virtual Meter Driver (C): Registers `/dev/smart_meter`, stores the pulse count, and handles ioctl requests.
- Driver Interface Header (C-compatible): Defines ioctl commands and shared data types for both kernel and user space.
- Pulse Simulator (C++): Generates pulse events and sends them to the driver.
- Analytics Agent (C++): Reads the count and calculates estimated kWh, power, and cost.
- Build and Documentation: Makefiles, README updates, execution instructions, diagrams, and stage reports.

Stage 2 Deliverables

- This Project Requirements Document.
- A documented project scope and module list.
- A development timeline through the 5 October 2026 deadline.
- Git history showing the Stage 2 work.

Development Timeline

Date	Planned work	Evidence
1 October 2026	Complete requirements and development plan; prepare Linux build tools.	Stage 2 PRD and Git commit
2 October 2026	Complete architecture, interface design, and UML diagrams.	Stage 3 design documents and Git commit
3 October 2026	Build the first driver and C++ prototype; demonstrate simulated pulse counting.	Prototype demonstration and Git commit
4 October 2026	Integrate, test, fix issues, and complete run instructions.	Test records, updated documentation, Git commit
5 October 2026	Final review, GitHub push, and project presentation preparation. Contact the trainer to arrange the 5-10 minute evaluation.	Final repository and presentation

Acceptance Criteria

- The driver builds and loads on the target Ubuntu Linux VM.
- `/dev/smart_meter` is created while the module is loaded and removed when it is unloaded.
- The C++ application can add simulated pulses and read the resulting count.
- The displayed energy and cost values match the documented formulas and configuration.
- Invalid input and unavailable-device errors are handled clearly.
- The README explains dependencies and how to build, run, and stop the project.
- Source code, documentation, and progress evidence are committed and pushed to GitHub.

Assumptions and Risks

- The Ubuntu VM kernel must have matching Linux kernel headers available.
- Driver development and module loading require administrator access.
- Kernel API differences may require small code adjustments for the installed Ubuntu kernel.
- Pulse input, power, energy, and cost are simulated estimates; they are not physical measurements.
- The project must be kept small enough to complete and demonstrate by the deadline.

Next Stage

Stage 3 - System Design & Architecture: define the architecture, ioctl data structures, device interactions, UML diagrams, Linux build environment, and implementation plan.

Stage 3 - System Design and Architecture

Design Summary

The prototype has two main software parts:

- A Linux kernel module written in C that provides a virtual smart-meter character device.
- A C++ command-line application that sends simulated pulse counts to the device, reads the accumulated count, and calculates estimated energy, power, and cost.

The project will run and be built in the Ubuntu Linux virtual machine. It uses no physical meter or sensor.

System Architecture

Rendered design diagrams are presented in the consolidated architecture section of this report.

Data Flow

- The user enters a simulated pulse amount in the C++ command-line application.
- The application opens `/dev/smart_meter`.
- It sends an `ioctl` request to add the simulated pulses.
- The driver validates the request and safely updates its counter.
- The application requests the current cumulative pulse count.
- The C++ analytics engine calculates and displays the estimates.

Main Components

Component	Language	Responsibility
Virtual meter driver	C	Register <code>/dev/smart_meter</code> , validate <code>ioctl</code> requests, protect and update the pulse counter, return readings
Shared <code>ioctl</code> interface	C-compatible header	Define <code>ioctl</code> command numbers and fixed-width data structures used by the driver and application
Smart-meter command-line application	C++	Accept user actions, communicate with the device, handle errors, and display readings
Analytics engine	C++	Convert pulse counts and elapsed time into estimated energy, power, and cost
Build files	Makefile	Build the kernel module and C++ application on Linux
Documentation	Markdown	Explain design, setup, execution, and project progress

Device Interface and Data Structures

The driver will use Linux's miscellaneous character device framework and expose the device as `/dev/smart_meter`.

The shared header will define three `ioctl` operations:

- `INJECT_PULSES`: Accept a positive pulse quantity and add it to the counter.
- `GET_COUNT`: Return the current cumulative pulse count.
- `RESET_COUNT`: Set the counter to zero for a new demonstration.

Planned data types:

- `pulse_request`: a fixed-width unsigned 32-bit pulse quantity.
- `pulse_count`: a fixed-width unsigned 64-bit cumulative count.
- `meter_config`: pulses per kWh and sample tariff, stored by the C++ application.

The kernel driver will protect counter updates and reads with a mutex. The driver will reject invalid requests and return a suitable error code.

Analytics Formulas

- Energy (kWh) = cumulative pulses / pulses per kWh
- Average power (W) = change in energy (kWh) x 3,600,000 / elapsed seconds
- Estimated cost = energy (kWh) x configured tariff per kWh

Initial demonstration configuration: 1000 pulses per kWh. The tariff is a sample value entered or configured for demonstration only. These values are estimates, not utility measurements or billing values.

C++ Class Design

Rendered design diagrams are presented in the consolidated architecture section of this report.

The kernel driver is written in C and is not represented as a C++ class. The application communicates with it through the shared `ioctl` interface.

Sequence Diagram

Rendered design diagrams are presented in the consolidated architecture section of this report.

Driver State Machine

Rendered design diagrams are presented in the consolidated architecture section of this report.

Planned Repository Structure

```
smart-meter-project/
  app/
    main.cpp
    smart_meter_device.cpp
    analytics_engine.cpp
  driver/
    smart_meter_driver.c
    Makefile
  include/
    smart_meter_ioctl.h
  docs/
    stage-1-project-introduction.md
    stage-2-project-requirements.md
    stage-3-system-design.md
  Makefile
  .gitignore
  README.md
```

Linux Development Environment

Development and execution will use Ubuntu 24.04 ARM64 inside UTM on the MacBook Air M2.

Required Linux tools:

- GCC C compiler and G++ C++ compiler
- GNU Make
- Git
- Linux kernel headers matching the running Ubuntu kernel

- Standard Linux kernel module tools, including `make`, `insmod`, `rmmmod`, and `dmesg`

Kernel-module loading and removal require administrator privileges. The C++ application will be run from the Ubuntu terminal.

Git Branching and Progress

- `main` is the stable branch used for the final GitHub submission.
- Create focused branches for substantial modules, such as `feature/kernel-driver` and `feature/cpp-analytics`.
- Commit small, understandable changes with messages describing the work.
- Merge a feature branch into `main` after reviewing the change.
- Push completed stage documentation and implementation to GitHub regularly.
- Keep progress evidence in the repository, such as diagrams, command output, and short demonstration notes.

Implementation Plan

- Confirm Ubuntu compiler, Make, Git, and matching kernel headers.
- Add the shared `ioctl` header and a minimal C kernel module.
- Build and load the module; confirm `/dev/smart_meter` appears.
- Add the C++ device wrapper and test pulse injection, reading, and reset.
- Add energy, estimated power, tariff, input validation, and clear errors.
- Document build and run instructions, then demonstrate the complete flow.

Stage 3 Completion Checklist

- ☐ Architecture and data-flow diagram documented.
- ☐ Driver, interface, application, and analytics responsibilities defined.
- ☐ Class, sequence, and state diagrams documented.
- ☐ Device interface and data structures specified.
- ☐ Ubuntu tools and build environment identified.
- ☐ Repository layout, branching approach, and implementation plan recorded.
- ☐ Stage 3 document committed and pushed to GitHub.

Next Stage

Stage 4 - Initial Implementation & Prototype: prepare the Linux build tools and implement the first working version of the driver and C++ application.

Stage 4 - Initial Implementation and Prototype

Stage Objective

Build an initial working prototype that connects a Linux kernel module to a C++ command-line analytics application.

Implemented Components

- Linux kernel module: Registers `/dev/smart_meter` and tracks a pulse count.
- IOCTL interface: Supports adding simulated pulses, reading the count, and resetting it.
- C++ device wrapper: Opens the device and sends the IOCTL requests.
- Analytics engine: Calculates estimated energy, average power, and sample cost.
- Command-line application: Provides options to simulate pulses, view analytics, reset the count, and exit.
- Build setup: The root `Makefile` builds the kernel module and C++ application.

Prototype Environment

- Operating system: Ubuntu Linux 24.04 ARM64 in UTM
- Kernel: 7.0.0-34-generic
- Application language: C++
- Driver language: C
- Meter constant: 1,000 pulses per kWh
- Sample tariff: INR 8 per kWh

Demonstrated Functionality

The prototype was built successfully. The driver created `/dev/smart_meter`, and the application was run with administrator privileges.

Demonstration:

- Simulated 100 pulses.
- Read a total of 100 pulses, estimated energy of 0.100 kWh, and estimated cost of INR 0.800.
- Simulated 1 additional pulse.
- Read a total of 101 pulses, estimated energy of 0.101 kWh, and estimated cost of INR 0.808.
- The application also displayed an average power estimate based on the time between readings.
- Exited the application successfully.

Issues and Resolutions

- The installed kernel headers did not provide `no_llseek`; removed that file-operation entry so the module compiled.
- The C++ device header was missing its opening `#ifndef` guard; added it to match the existing `#define` and `#endif`.
- Compiler naming, `pahole`, and BTF messages appeared during the kernel build. They did not prevent the module from building.

Current Limitations

- Pulses are simulated through the application; no physical meter is connected.
- Energy and cost are estimates based on the configured meter constant and sample tariff.
- The average power estimate depends on the time between readings.
- The application currently requires administrator privileges to access the device.

Next Stage

Stage 5 will add and document unit, integration, and system testing; resolve issues found; and improve reliability and documentation.

Stage 5 - Testing, Integration and Improvement

Stage Objective

Check that the Linux kernel module and C++ application build and work together correctly. Record test results, fix any issues found, and update the project documentation.

Test Environment

- Operating system: Ubuntu Linux 24.04 ARM64 in UTM
- Kernel: Record the output of `uname -r` when testing
- Driver language: C
- Application language: C++
- Hardware: Simulated pulse input; no physical meter is connected

Planned Tests

ID	Test	Expected result
T1	Build the driver and C++ application with <code>make</code>	Both build successfully
T2	Load the kernel module	The module loads and <code>/dev/smart_meter</code> exists
T3	Start the application	The menu appears without an error
T4	Add 100 pulses and read analytics	Pulse count is 100; energy is 0.100 kWh; estimated cost is INR 0.800
T5	Add 1 more pulse and read analytics	Pulse count is 101; energy is 0.101 kWh; estimated cost is INR 0.808
T6	Reset the count and read it again	Pulse count is reset to 0
T7	Try an invalid menu choice or pulse input	The application handles invalid input without crashing
T8	Exit and unload the module	The application exits and the module unloads cleanly

Test Result Record

For each test, record:

- Test ID and date
- Command or menu actions used
- Expected result
- Actual result
- Pass or fail
- Any issue found and how it was resolved

Do not mark a test as passed until it has been performed.

Known Limitations

- Pulse input is simulated by the software; no physical meter is connected.

- Energy and cost are estimates using the configured meter constant and sample tariff.
- Average power depends on the time between readings.
- Device access currently requires administrator privileges.

Next Step

Run the planned tests in Ubuntu, record their actual results, fix any issues, and then update the README and project instructions if needed.

Test Results - 2 October 2026

ID	Result	Observed outcome
T1	Pass	<code>make clean</code> && <code>make</code> rebuilt the kernel module and C++ application successfully.
T2	Pass	<code>smart_meter_driver</code> was loaded and <code>/dev/smart_meter</code> existed.
T3	Pass	The application started and displayed its menu.
T4	Pass	After reset, 100 pulses produced a count of 100, energy of 0.100 kWh, and estimated cost of INR 0.800.
T5	Pass	One additional pulse produced a count of 101, energy of 0.101 kWh, and estimated cost of INR 0.808.
T6	Pass	Resetting the count and reading analytics showed 0 pulses, 0.000 kWh, and INR 0.000.
T7	Pass	Menu choice 9 was rejected with a helpful message. Pulse input 0 was rejected with the allowed range. The application remained open.
T8	Pass	The application exited. The driver unloaded; <code>lsmod</code> showed no matching module, and <code>/dev/smart_meter</code> no longer existed.

Issues Observed and Improvement Notes

- The clean build showed compiler-name, pahole, and BTF messages. The build still succeeded; BTF generation was skipped because `vmlinux` was unavailable.
- Average power changed substantially between readings (for example, 5517.960 W and 43.199 W). It depends on the elapsed time between readings, so very short intervals can produce high estimates. This remains a limitation to explain during the demonstration.
- The device file is owned by `root` and the application was run with `sudo`. A future improvement could provide a controlled non-root access rule.

Stage 5 Progress

The clean build, driver availability, application menu, pulse calculations, reset, invalid-input handling, application exit, and driver unload were checked in the Ubuntu VM. The observed readings matched the configured meter constant and sample tariff.

Stage 6 - Final Project Report

Project Title

Smart Energy Smart-Meter Pulse Counter & Analytics Agent

Project Summary

This individual capstone is a software-only Linux smart-meter prototype. A C Linux kernel module exposes a virtual pulse-counting device. A C++ application sends simulated pulses to the driver, reads the pulse count, and estimates energy use, average power, and cost.

No physical meter or sensor is connected. The project demonstrates Linux device-driver concepts, system programming, C++, software architecture, testing, and Git-based development.

Objectives

- Build and load an external Linux kernel module.
- Expose a device interface at `/dev/smart_meter`.
- Communicate between user space and the driver using IOCTL requests.
- Implement the command-line application and analytics in C++.
- Document the architecture, requirements, implementation, and test results.
- Maintain the source code and documentation in GitHub.

Architecture

The C++ application uses `SmartMeterDevice` to communicate with `/dev/smart_meter`. The device file is handled by the Linux kernel module. The driver protects the pulse counter with a mutex and supports pulse injection, count reading, and reset operations through the shared IOCTL definitions.

The `AnalyticsEngine` calculates energy, average power, and estimated cost from the pulse count, meter constant, tariff, and elapsed time. Architecture and UML diagrams are included in `README.md`.

Implementation

- Kernel driver (C): Registers a miscellaneous character device and handles the pulse-count IOCTL operations.
- Device wrapper (C++): Opens the device and sends the driver requests.
- Analytics engine (C++): Calculates energy, average power, and sample cost.
- Command-line application (C++): Simulates pulses and displays readings.
- Build system: The root `Makefile` builds the driver and application and runs the analytics unit tests.
- Documentation: Project stages, architecture, and test results are recorded in `docs/`.

Final Implementation, Demonstration and Outcomes

This consolidated final section completes the build and demonstration record using the verified implementation and test evidence documented in Stages 4 and 5.

Final implementation and demonstration outcome

The Linux kernel module and C++ application built successfully in Ubuntu Linux 24.04 ARM64. The driver loaded and registered `/dev/smart_meter`. The application simulated pulse input, displayed cumulative counts and energy/cost estimates, reset the count, rejected invalid input, and exited. The recorded manual examples produced 100 pulses = 0.100 kWh = INR 0.800 and 101 pulses = 0.101 kWh = INR 0.808.

Testing outcome

The eight analytics unit checks passed. Manual integration records cover build, driver loading/device appearance, application menu, pulse simulation, analytics output, reset, invalid inputs, application exit, and driver unload. The unit tests exercise calculation functions, not the kernel module.

Achievements

The project demonstrates C++ user-space programming, Linux system calls through a character-device interface, IOCTL communication, kernel module lifecycle, mutex-protected shared state, energy/power/cost formulas, Make-based build steps, Git-based development, and staged documentation.

Limitations and possible improvements

Pulse events are simulated. The cumulative count remains only in kernel memory while the module is loaded and is lost on module unload or Linux restart. The sample tariff and meter constant are fixed in the current application. Usage classification, alerts and persistent file/database logging are not implemented. Future work may add configurable rates, timestamped storage, summaries and documented usage thresholds.

Evaluation and submission

The instructor requirements call for a 5-10 minute evaluation with the trainer at a mutually convenient time. The project repository is available at [GitHub - smart-energy-smart-meter-pulse-counter-analytics-agent](#).

Appendix A - Build, Run and Test Quick Reference

Use a terminal inside the Ubuntu virtual machine. Run commands from the project directory and in this order for a demonstration.

Enter the project directory

```
cd ~/smart-meter-project
```

Build the kernel module and C++ program

```
make
```

Load the kernel driver

```
sudo insmod driver/smart_meter_driver.ko
```

Launch the app

```
sudo ./smart_meter_agent
```

Exit through menu option 4, then unload the driver

```
sudo rmmod smart_meter_driver
```

Run analytics tests (driver is not needed)

```
make unit-test
```

Why unload the module?

After the application exits and closes `/dev/smart_meter`, `sudo rmmod smart_meter_driver` asks Linux to remove the kernel module. The driver releases its registered device and its in-memory counter disappears. This is normal cleanup, not deletion of project files. Run the unit tests after this step; the analytics unit test executable uses only the C++ calculation source and does not load the kernel driver.

Example unit-test output

```
PASS: 100 pulses at 1000 pulses/kWh = 0.1 kWh
PASS: zero pulses = 0 kWh
PASS: zero meter constant returns 0 kWh
PASS: 10 pulses over one hour = 10 W
PASS: zero elapsed time returns 0 W
PASS: decreasing pulse count returns 0 W
PASS: 0.1 kWh at INR 8/kWh = INR 0.8
PASS: negative energy returns INR 0
All analytics unit tests passed.
```

Recorded integration outcomes

Check	Observed outcome
Clean build	Kernel module and C++ application built successfully; compiler/pahole/BTF notices were non-fatal.
Driver lifecycle	Driver loaded; <code>/dev/smart_meter</code> appeared; after unload, module and device entry were absent.
Pulse and analytics	100 pulses -> 0.100 kWh -> INR 0.800; 101 pulses -> 0.101 kWh -> INR 0.808.
Reset / invalid inputs	Reset returned count to zero; menu choice 9 and pulse value 0 were rejected without a crash.
Analytics unit tests	All eight checks passed.

Appendix B - Complete Project Source Code

The listings below reproduce the project source and build files. Generated kernel build products (.o, .ko, .mod.c, command files and binaries) are excluded because they are generated by the Linux build and are not the maintained source. File behavior is summarized before each listing.

Makefile

Builds the kernel module and C++ application; includes clean and analytics unit-test targets.

```
KERNEL_BUILD := /lib/modules/$(shell uname -r)/build
PROJECT_DIR := $(CURDIR)
CXX := g++
CXXFLAGS := -std=c++17 -Wall -Wextra -Wpedantic -Iapp -Iinclude

APP_SOURCES := app/main.cpp app/smart_meter_device.cpp app/analytics_engine.cpp
APP_TARGET := smart_meter_agent

.PHONY: all driver app clean

all: driver app

driver:
  $(MAKE) -C $(KERNEL_BUILD) M=$(PROJECT_DIR)/driver modules

app: $(APP_TARGET)

$(APP_TARGET): $(APP_SOURCES) include/smart_meter_ioctl.h
  $(CXX) $(CXXFLAGS) $(APP_SOURCES) -o $@

clean:
  $(MAKE) -C $(KERNEL_BUILD) M=$(PROJECT_DIR)/driver clean
  rm -f $(APP_TARGET) tests/analytics_engine_test

.PHONY: unit-test

unit-test: tests/analytics_engine_test
  ./tests/analytics_engine_test

tests/analytics_engine_test: tests/analytics_engine_test.cpp app/analytics_engine.cpp app/
analytics_engine.hpp
  $(CXX) $(CXXFLAGS) tests/analytics_engine_test.cpp app/analytics_engine.cpp -o $@
```

driver/Makefile

Names the external kernel-module object for the Linux Kbuild system.

```
obj-m += smart_meter_driver.o
```

include/smart_meter_ioctl.h

Defines the IOCTL request identifiers and fixed-width pulse data shared by user and kernel space.

```
#ifndef SMART_METER_IOCTL_H
#define SMART_METER_IOCTL_H

#include <linux/ioctl.h>
#include <linux/types.h>

/*
 * Shared ioctl interface for the Smart Meter Linux driver and C++ application.
 * Keep the command numbers and data types consistent in both components.
 */
#define SMART_METER_IOC_MAGIC 'S'

#define SMART_METER_IOC_INJECT_PULSES \
    _IOW(SMART_METER_IOC_MAGIC, 1, __u32)

#define SMART_METER_IOC_GET_COUNT \
    _IOR(SMART_METER_IOC_MAGIC, 2, __u64)

#define SMART_METER_IOC_RESET_COUNT \
    _IO(SMART_METER_IOC_MAGIC, 3)

#endif /* SMART_METER_IOCTL_H */
```

driver/smart_meter_driver.c

Registers the miscellaneous device, maintains the protected counter, validates IOCTL operations and cleans up on module removal.

```
#include <linux/errno.h>
#include <linux/fs.h>
#include <linux/miscdevice.h>
#include <linux/module.h>
#include <linux/mutex.h>
#include <linux/uaccess.h>

#include "../include/smart_meter_ioctl.h"

#define DEVICE_NAME "smart_meter"

static DEFINE_MUTEX(smart_meter_mutex);
static __u64 smart_meter_pulse_count;

static long smart_meter_ioctl(struct file *file,
                             unsigned int command,
                             unsigned long argument)
{
    __u32 pulses;
    __u64 count;

    (void)file;

    if (_IOC_TYPE(command) != SMART_METER_IOC_MAGIC)
        return -ENOTTY;

    switch (command) {
    case SMART_METER_IOC_INJECT_PULSES:
        if (copy_from_user(&pulses, (void __user *)argument,
                          sizeof(pulses)))
            return -EFAULT;

        if (pulses == 0)
            return -EINVAL;

        mutex_lock(&smart_meter_mutex);

        if (smart_meter_pulse_count > (~(__u64)0 - pulses)) {
            mutex_unlock(&smart_meter_mutex);
            return -EOVERFLOW;
        }

        smart_meter_pulse_count += pulses;
        mutex_unlock(&smart_meter_mutex);
        return 0;

    case SMART_METER_IOC_GET_COUNT:
        mutex_lock(&smart_meter_mutex);
        count = smart_meter_pulse_count;
        mutex_unlock(&smart_meter_mutex);

        if (copy_to_user((void __user *)argument, &count, sizeof(count)))
            return -EFAULT;

        return 0;

    case SMART_METER_IOC_RESET_COUNT:
        mutex_lock(&smart_meter_mutex);
        smart_meter_pulse_count = 0;
        mutex_unlock(&smart_meter_mutex);
        return 0;

    default:
        return -ENOTTY;
    }
}

static const struct file_operations smart_meter_file_operations = {
    .owner = THIS_MODULE,
    .unlocked_ioctl = smart_meter_ioctl,
};

static struct miscdevice smart_meter_device = {
    .minor = MISC_DYNAMIC_MINOR,
    .name = DEVICE_NAME,
    .fops = &smart_meter_file_operations,
    .mode = 0660,
};
```

```
static int __init smart_meter_init(void)
{
    int result = misc_register(&smart_meter_device);

    if (result)
        pr_err("smart_meter: failed to register device: %d\n", result);
    else
        pr_info("smart_meter: registered /dev/%s\n", DEVICE_NAME);

    return result;
}

static void __exit smart_meter_exit(void)
{
    misc_deregister(&smart_meter_device);
    pr_info("smart_meter: unregistered /dev/%s\n", DEVICE_NAME);
}

module_init(smart_meter_init);
module_exit(smart_meter_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Smart Meter Capstone Project");
MODULE_DESCRIPTION("Software-only virtual smart-meter pulse counter");
MODULE_VERSION("1.0");
```

app/smart_meter_device.hpp

Declares the RAIL-style device wrapper and pulse injection/read/reset interface.

```
#ifndef SMART_METER_DEVICE_HPP
#define SMART_METER_DEVICE_HPP

#include <cstdint>

class SmartMeterDevice {
public:
    SmartMeterDevice();
    ~SmartMeterDevice();

    SmartMeterDevice(const SmartMeterDevice&) = delete;
    SmartMeterDevice& operator=(const SmartMeterDevice&) = delete;

    bool openDevice();
    bool injectPulses(std::uint32_t pulses);
    bool getPulseCount(std::uint64_t& count) const;
    bool resetCount() const;
    void closeDevice();
    bool isOpen() const noexcept;

private:
    int file_descriptor_;
};

#endif
```

app/smart_meter_device.cpp

Opens /dev/smart_meter and sends the shared IOCTL requests.

```
#include "smart_meter_device.hpp"

#include "../include/smart_meter_ioctl.h"

#include <fcntl.h>
#include <iostream>
#include <sys/ioctl.h>
#include <unistd.h>

SmartMeterDevice::SmartMeterDevice()
    : file_descriptor_(-1)
{
}

SmartMeterDevice::~SmartMeterDevice()
{
    closeDevice();
}

bool SmartMeterDevice::openDevice()
{
    if (isOpen())
        return true;

    file_descriptor_ = ::open("/dev/smart_meter", O_RDWR | O_CLOEXEC);
    if (file_descriptor_ < 0) {
        perror("Could not open /dev/smart_meter");
        return false;
    }

    return true;
}

bool SmartMeterDevice::injectPulses(std::uint32_t pulses)
{
    if (!isOpen()) {
        std::cerr << "Device is not open.\n";
        return false;
    }

    if (pulses == 0) {
        std::cerr << "Pulse quantity must be greater than zero.\n";
        return false;
    }

    if (::ioctl(file_descriptor_, SMART_METER_IOC_INJECT_PULSES, &pulses) < 0) {
        perror("Could not inject pulses");
        return false;
    }

    return true;
}

bool SmartMeterDevice::getPulseCount(std::uint64_t& count) const
{
    if (!isOpen()) {
        std::cerr << "Device is not open.\n";
        return false;
    }

    if (::ioctl(file_descriptor_, SMART_METER_IOC_GET_COUNT, &count) < 0) {
        perror("Could not read pulse count");
        return false;
    }

    return true;
}

bool SmartMeterDevice::resetCount() const
{
    if (!isOpen()) {
        std::cerr << "Device is not open.\n";
        return false;
    }

    if (::ioctl(file_descriptor_, SMART_METER_IOC_RESET_COUNT, 0) < 0) {
        perror("Could not reset pulse count");
        return false;
    }
}
```

```
        return true;
    }

    void SmartMeterDevice::closeDevice()
    {
        if (file_descriptor_ >= 0) {
            if (::close(file_descriptor_) < 0)
                perror("Could not close /dev/smart_meter");

            file_descriptor_ = -1;
        }
    }

    bool SmartMeterDevice::isOpen() const noexcept
    {
        return file_descriptor_ >= 0;
    }
}
```

app/analytics_engine.hpp

Declares energy, power and cost calculation functions.

```
#ifndef ANALYTICS_ENGINE_HPP
#define ANALYTICS_ENGINE_HPP

#include <cstdint>

class AnalyticsEngine {
public:
    static double calculateEnergyKwh(
        std::uint64_t pulse_count,
        double pulses_per_kwh) noexcept;

    static double calculateAveragePowerWatts(
        std::uint64_t previous_pulse_count,
        std::uint64_t current_pulse_count,
        double pulses_per_kwh,
        double elapsed_seconds) noexcept;

    static double calculateEstimatedCost(
        double energy_kwh,
        double tariff_per_kwh) noexcept;
};

#endif
```

app/analytics_engine.cpp

Implements calculation formulas and rejects invalid calculation inputs.

```
#include "analytics_engine.hpp"

double AnalyticsEngine::calculateEnergyKwh(
    std::uint64_t pulse_count,
    double pulses_per_kwh) noexcept
{
    if (pulses_per_kwh <= 0.0)
        return 0.0;

    return static_cast<double>(pulse_count) / pulses_per_kwh;
}

double AnalyticsEngine::calculateAveragePowerWatts(
    std::uint64_t previous_pulse_count,
    std::uint64_t current_pulse_count,
    double pulses_per_kwh,
    double elapsed_seconds) noexcept
{
    if (pulses_per_kwh <= 0.0 || elapsed_seconds <= 0.0)
        return 0.0;

    if (current_pulse_count < previous_pulse_count)
        return 0.0;

    const std::uint64_t pulse_difference =
        current_pulse_count - previous_pulse_count;

    const double energy_difference_kwh =
        static_cast<double>(pulse_difference) / pulses_per_kwh;

    return energy_difference_kwh * 3600000.0 / elapsed_seconds;
}

double AnalyticsEngine::calculateEstimatedCost(
    double energy_kwh,
    double tariff_per_kwh) noexcept
{
    if (energy_kwh < 0.0 || tariff_per_kwh < 0.0)
        return 0.0;

    return energy_kwh * tariff_per_kwh;
}
```

app/main.cpp

Implements menu actions, simulation, validation, analytics display and elapsed-time power estimates.

```
#include "analytics_engine.hpp"
#include "smart_meter_device.hpp"

#include <chrono>
#include <cstdlib>
#include <iomanip>
#include <iostream>
#include <limits>

int main()
{
    SmartMeterDevice device;

    if (!device.openDevice()) {
        std::cerr << "Make sure the smart-meter driver is loaded and run "
                    "this application with sudo.\n";
        return 1;
    }

    constexpr double pulses_per_kwh = 1000.0;
    constexpr double sample_tariff_per_kwh = 8.0;

    bool has_previous_reading = false;
    std::uint64_t previous_pulse_count = 0;
    std::chrono::steady_clock::time_point previous_reading_time;

    while (true) {
        std::cout << "\n=== Smart Meter Analytics Agent ===\n"
                  << "1. Simulate pulses\n"
                  << "2. Show meter analytics\n"
                  << "3. Reset pulse count\n"
                  << "4. Exit\n"
                  << "Choose an option: ";

        int choice = 0;
        if (!(std::cin >> choice)) {
            if (std::cin.eof())
                break;

            std::cin.clear();
            std::cin.ignore(
                std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "Enter a number from 1 to 4.\n";
            continue;
        }

        if (choice == 1) {
            std::uint64_t requested_pulses = 0;
            std::cout << "Enter the number of pulses to simulate: ";

            if (!(std::cin >> requested_pulses)) {
                std::cin.clear();
                std::cin.ignore(
                    std::numeric_limits<std::streamsize>::max(), '\n');
                std::cout << "Enter a whole number.\n";
                continue;
            }

            if (requested_pulses == 0 ||
                requested_pulses >
                    std::numeric_limits<std::uint32_t>::max()) {
                std::cout << "Enter a value from 1 to "
                            << std::numeric_limits<std::uint32_t>::max()
                            << ".\n";
                continue;
            }

            if (device.injectPulses(
                static_cast<std::uint32_t>(requested_pulses))) {
                std::cout << "Simulated "
                            << requested_pulses << " pulse(s).\n";
            }
        } else if (choice == 2) {
            std::uint64_t pulse_count = 0;
            if (!device.getPulseCount(pulse_count))
                continue;

            const auto now = std::chrono::steady_clock::now();
            const double energy_kwh =
                AnalyticsEngine::calculateEnergyKwh(
```

```

        pulse_count, pulses_per_kwh);
const double estimated_cost =
    AnalyticsEngine::calculateEstimatedCost(
        energy_kwh, sample_tariff_per_kwh);

std::cout << std::fixed << std::setprecision(3)
    << "\nPulse count: " << pulse_count << '\n'
    << "Estimated energy: " << energy_kwh << " kWh\n"
    << "Sample tariff: INR "
    << sample_tariff_per_kwh << " per kWh\n"
    << "Estimated cost: INR "
    << estimated_cost << '\n';

if (has_previous_reading) {
    const double elapsed_seconds =
        std::chrono::duration<double>(
            now - previous_reading_time).count();

    const double average_power_watts =
        AnalyticsEngine::calculateAveragePowerWatts(
            previous_pulse_count,
            pulse_count,
            pulses_per_kwh,
            elapsed_seconds);

    std::cout << "Estimated average power since the last "
        "reading: "
        << average_power_watts << " W\n";
} else {
    std::cout << "Take another reading to estimate average "
        "power.\n";
}

previous_pulse_count = pulse_count;
previous_reading_time = now;
has_previous_reading = true;
} else if (choice == 3) {
    if (device.resetCount()) {
        std::cout << "Pulse count reset to zero.\n";
        has_previous_reading = false;
    }
} else if (choice == 4) {
    std::cout << "Exiting the analytics agent.\n";
    break;
} else {
    std::cout << "Choose an option from 1 to 4.\n";
}
}

return 0;
}

```

tests/analytics_engine_test.cpp

Checks eight standard and edge-case calculations without loading the Linux driver.

```
#include "analytics_engine.hpp"

#include <cmath>
#include <iostream>

namespace {
int failures = 0;

void expectNear(const char* test_name, double actual, double expected)
{
    constexpr double tolerance = 1e-9;

    if (std::fabs(actual - expected) > tolerance) {
        std::cerr << "FAIL: " << test_name
            << " (expected " << expected
            << ", got " << actual << ")\n";
        ++failures;
    } else {
        std::cout << "PASS: " << test_name << '\n';
    }
}
} // namespace

int main()
{
    expectNear(
        "100 pulses at 1000 pulses/kWh = 0.1 kWh",
        AnalyticsEngine::calculateEnergyKwh(100, 1000.0),
        0.1);

    expectNear(
        "zero pulses = 0 kWh",
        AnalyticsEngine::calculateEnergyKwh(0, 1000.0),
        0.0);

    expectNear(
        "zero meter constant returns 0 kWh",
        AnalyticsEngine::calculateEnergyKwh(100, 0.0),
        0.0);

    expectNear(
        "10 pulses over one hour = 10 W",
        AnalyticsEngine::calculateAveragePowerWatts(
            100, 110, 1000.0, 3600.0),
        10.0);

    expectNear(
        "zero elapsed time returns 0 W",
        AnalyticsEngine::calculateAveragePowerWatts(
            100, 110, 1000.0, 0.0),
        0.0);

    expectNear(
        "decreasing pulse count returns 0 W",
        AnalyticsEngine::calculateAveragePowerWatts(
            110, 100, 1000.0, 3600.0),
        0.0);

    expectNear(
        "0.1 kWh at INR 8/kWh = INR 0.8",
        AnalyticsEngine::calculateEstimatedCost(0.1, 8.0),
        0.8);

    expectNear(
        "negative energy returns INR 0",
        AnalyticsEngine::calculateEstimatedCost(-0.1, 8.0),
        0.0);

    if (failures != 0) {
        std::cerr << failures << " unit test(s) failed.\n";
        return 1;
    }

    std::cout << "All analytics unit tests passed.\n";
    return 0;
}
```

Appendix C - Submission Notes

The GitHub repository contains the source, README, six stage documents, UML diagrams in Mermaid source, and testing records. This PDF consolidates the project narrative and includes the maintained source/build files for evaluator convenience.

GitHub repository: [smart-energy-smart-meter-pulse-counter-analytics-agent](#)

The project is an educational software prototype. It does not connect to a physical meter, persist readings, classify usage, or produce billing-grade measurements.

End of report